

SENA · ADSO 2026

Clase 33 — Diseño Técnico del Proyecto

Diseño Técnico del Proyecto

Texticode — Sistema de Gestión Textil

API de Carga de Trabajo

Aprendices

Kevin Stevan Wagner

Kevin Andrés Castro

Andrés Felipe Lancheros

Camilo Andrés Tibambre

Instructor

Ing. Edwin Marín

Fecha: Mayo 2026

Programa: ADSO 2026

1. Arquitectura 3 Capas

Texticode implementa una arquitectura REST desacoplada en 3 capas. El frontend nunca accede directamente a la base de datos — toda comunicación ocurre vía HTTP con autenticación por API Key.

Capa	Detalle
FRONTEND	Vue.js 3 · EficienciaOperarios.vue · Fetch HTTP · VITE_API_KEY en .env
BACKEND	Node.js + Express · routes/carga_trabajo.js · middleware apiKey.js · Lógica de negocio
BASE DE DATOS	Supabase PostgreSQL (192.168.10.35) · tablas: usuario, rol, orden_produccion

Flujo de datos — detectar sobrecarga y reasignar órdenes

Capa	Acción
Frontend	EficienciaOperarios.vue → <code>fetch('http://192.168.10.35:3001/api/carga-trabajo', { headers: { 'x-api-key': KEY } })</code>
Backend · Auth	Middleware apiKey.js verifica x-api-key contra API_KEY_EFICIENCIA en .env
Backend · Business	carga_trabajo.js clasifica operarios: sobrecargado (>5 órdenes) / normal (3-5) / disponible (≤2)
Persistence	pg.Pool ejecuta SELECT + JOIN en Supabase PostgreSQL (Session Pooler 6543)
Database	PostgreSQL devuelve registros con garantías ACID
Response	200 OK + { ok: true, resumen: { sobrecargados, normales, disponibles }, data: [...] }

2. Contrato Técnico API

Los 9 contratos de la API de Carga de Trabajo, documentados con la URL completa de cada endpoint y espacio reservado para la evidencia de pruebas en Thunder Client.

2.1 GET /api/carga-trabajo

GET**/api/carga-trabajo***Lista todos los operarios activos clasificados por carga en tiempo real.*

REQUEST

```
GET /api/carga-trabajo HTTP/1.1
Host: 192.168.10.35:3001
x-api-key: texticode-2026

// Query params opcionales
?estado=sobrecargado|normal|disponible
```

URL completa

```
http://192.168.10.35:3001/api/carga-trabajo
```

EVIDENCIA THUNDER CLIENT

The screenshot shows the Thunder Client interface with the following details:

- Query:** GET `http://192.168.10.35:3001/api/carga-trabajo`
- Status:** 200 OK, Size: 1.39 KB, Time: 1.05 s
- Response (JSON):**

```
{
  "ok": true,
  "filtro_aplicado": null,
  "resumen": {
    "total_operarios": 5,
    "sobrecargados": 4,
    "normales": 1,
    "disponibles": 0
  },
  "data": [
    {
      "Id_Usuario": 52,
      "Nombre_Completo": "Stevan Wagner",
      "Nombre_Usuario": "Wagner2",
      "Correo": "stevanwagner99@gmail.com",
      "Telefono": "+57 322 442 2134",
      "ordenes_activas": 20,
      "fases_activas": 10,
      "ordenes_vencidas": 12,
      "ordenes_alta_prioridad": 7,
      "estado_carga": "sobrecargado"
    },
    {
      "Id_Usuario": 50,
      "Nombre_Completo": "Andres Castro",
      "Nombre_Usuario": "Kevin2",
      "Correo": "vertorigo71@gmail.com",
      "Telefono": "+57 316 5631588",
      "ordenes_activas": 11,
      "fases_activas": 7,
      "ordenes_vencidas": 8,
      "ordenes_alta_prioridad": 5,
      "estado_carga": "sobrecargado"
    },
    {
      "Id_Usuario": 43,
      "Nombre_Completo": "Andres Montaña ",
      "Nombre_Usuario": "Montaña2",
      "Correo": "uscamilotibambre@gmail.com",
      "Telefono": "+57 320 3590156",
      "ordenes_activas": 11,
      "fases_activas": 7,
      "ordenes_vencidas": 8,
      "ordenes_alta_prioridad": 5,
      "estado_carga": "sobrecargado"
    }
  ]
}
```

192.168.10.35:3001/api/ca. X

GET

http://192.168.10.35:3001/api/carga-trabajo

Send

Query

Headers 3

Auth

Body

Tests

Pre Run

HTTP Headers

☐ Raw

☒ Accept

/*

☒ User-Agent

Thunder Client (https://www.thunderclient.com)

☒ x-api-key

texticode-202

☐ header

value

Status: 401 Unauthorized

Size: 84 Bytes

Time: 11 ms

Response

Headers 7

Cookies

Results

Docs

</>

≡

1 {

2 "ok": false,

3 "mensaje": "Acceso denegado. Se requiere API Key en el header x-api-key"

4 }

2.2 GET /api/carga-trabajo/sugerencias

GET**/api/carga-trabajo/sugerencias***Genera sugerencias automáticas de reasignación priorizando órdenes vencidas y de alta prioridad.*

REQUEST

```
GET /api/carga-trabajo/sugerencias HTTP/1.1
Host: 192.168.10.35:3001
x-api-key: texticode-2026

// Sin body ni parámetros – calcula automáticamente
```

URL completa

```
http://192.168.10.35:3001/api/carga-trabajo/sugerencias
```

EVIDENCIA THUNDER CLIENT

The screenshot shows the Thunder Client interface with a GET request to `http://192.168.10.35:3001/api/carga-trabajo/sugerencias`. The status is **200 OK**, size is **108 Bytes**, and time is **248 ms**. The response body is a JSON object:

```
{
  "ok": true,
  "mensaje": "Hay operarios sobrecargados pero ninguno tiene capacidad disponible",
  "sugerencias": []
}
```

The screenshot shows the Thunder Client interface with a GET request to `http://192.168.10.35:3001/api/carga-trabajo/sugerencias`. The status is **401 Unauthorized**, size is **84 Bytes**, and time is **3 ms**. The response body is a JSON object:

```
{
  "ok": false,
  "mensaje": "Acceso denegado. Se requiere API Key en el header x-api-key"
}
```

The Headers tab is selected, showing the following headers:

Header	Value
Accept	*/*
User-Agent	Thunder Client (https://www.thunderclient.com)
x-api-key	texticode-2026
header	value

2.3 GET /api/carga-trabajo/operarios/:id

GET**/api/carga-trabajo/operarios/:id***Devuelve detalle completo de carga de un operario específico con sus órdenes activas.*

REQUEST

```
GET /api/carga-trabajo/operarios/:id HTTP/1.1
Host: 192.168.10.35:3001
x-api-key: texticode-2026

// :id -> Id_Usuario del operario (entero positivo)
```

URL completa

```
http://192.168.10.35:3001/api/carga-trabajo/operarios/:id
```

EVIDENCIA THUNDER CLIENT

The screenshot shows the Thunder Client interface with a GET request to `http://192.168.10.35:3001/api/carga-trabajo/operarios/43`. The response status is **200 OK** with a size of **2.99 KB** and a time of **1.20 s**. The response body is a JSON object containing user details and active orders.

```
1 {
2   "ok": true,
3   "data": {
4     "Id_Usuario": 43,
5     "Nombre_Completo": "Andres Montaña ",
6     "Nombre_Usuario": "Montaña2",
7     "Correo": "uscamilotibambre@gmail.com",
8     "Telefono": "+57 320 3590156",
9     "ordenes_activas": 11,
10    "fases_activas": 7,
11    "ordenes_vencidas": 4,
12    "ordenes_alta_prioridad": 3,
13    "estado_carga": "sobrecargado",
14    "ordenes_activas_detalle": [
15      {
16        "Id_Orden_Operario": null,
17        "tipo": "orden",
18        "Id_Orden": 162,
19        "Id_Operario": 43,
20        "Numero_Fase": null,
21        "Descripcion_Fase": null,
22        "estado_item": "Retrasada",
23        "Producto": "Cobijas",
24        "Prioridad": "Alta",
25        "Fecha_Limite": "2026-09-10T05:00:00.000Z",
26        "vencida": true
27      },
28    ]
29  }
30 }
```

The screenshot shows the Thunder Client interface with a GET request to `http://192.168.10.35:3001/api/carga-trabajo/operarios/100000`. The response status is **404 Not Found** with a size of **63 Bytes** and a time of **1.11 s**. The response body is a JSON object indicating that no operator was found for the given ID.

```
1 {
2   "ok": false,
3   "mensaje": "No se encontró operario con id 100000"
4 }
```

2.4 POST /api/carga-trabajo/reasignar

POST**/api/carga-trabajo/reasignar***Reasigna una orden de producción de un operario a otro.*

REQUEST

```
POST /api/carga-trabajo/reasignar HTTP/1.1
Host: 192.168.10.35:3001
x-api-key: texticode-2026
Content-Type: application/json

{
  "Id_Orden": 13,
  "Id_Operario_Destino": 27
}
```

// Ambos campos son requeridos.
// La orden debe estar En Proceso o Pausado.
// El operario destino debe ser operario activo.

URL completa

```
http://192.168.10.35:3001/api/carga-trabajo/reasignar
```

EVIDENCIA THUNDER CLIENT

The screenshot displays the Thunder Client interface. The top bar shows the URL `http://192.168.10.35:3001/api/carga-trabajo/reasignar` and the method `POST`. The `Send` button is visible. Below the URL bar, the `Body` tab is selected, showing the JSON content: `{ "Id_Orden": 166, "Id_Operario_Destino": 47 }`. The `Response` tab is also visible, showing the status `200 OK`, size `265 Bytes`, and time `383 ms`. The response body is a JSON object: `{ "ok": true, "mensaje": "Fase 4 de la orden #179 reasignada a Andres Montaña ", "data": { "tipo": "fase", "Id_Orden_Operario": 42, "Id_Orden": 179, "Numero_Fase": 4, "Producto": "Camisetas oversize ", "operario_anterior": 43, "operario_nuevo": { "id": 43, "nombre": "Andres Montaña " } } }`.

The screenshot shows a REST client interface with a POST request to `http://192.168.10.35:3001/api/carga-trabajo/reasignar`. The request body is empty. The response status is **400 Bad Request** with a size of 95 Bytes and a time of 3 ms. The response body is a JSON object:

```
{  "ok": false,  "mensaje": "Se requiere Id_Orden_Operario (fase) o Id_Orden, y Id_Operario_Destino"}
```

The screenshot shows the same REST client interface with a POST request to `http://192.168.10.35:3001/api/carga-trabajo/reasignar`. The request body is a JSON object:

```
{  "Id_Orden": 1664,  "Id_Operario_Destino": 47}
```

The response status is **404 Not Found** with a size of 69 Bytes and a time of 1.12 s. The response body is a JSON object:

```
{  "ok": false,  "mensaje": "No se encontró una orden activa con id 1664"}
```

2.5 POST /api/carga-trabajo/aplicar-sugerencias

POST**/api/carga-trabajo/aplicar-sugerencias***Calcula y ejecuta automáticamente todas las reasignaciones sugeridas sin intervención manual.*

REQUEST

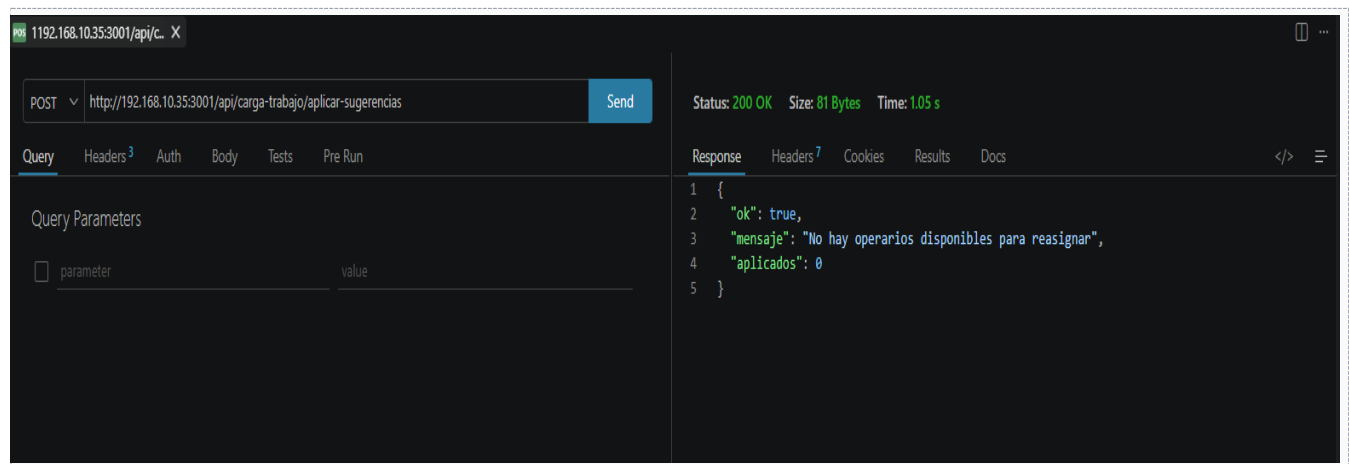
```
POST /api/carga-trabajo/aplicar-sugerencias HTTP/1.1
Host: 192.168.10.35:3001
x-api-key: texticode-2026

// Sin body – calcula y aplica automáticamente
```

URL completa

```
http://192.168.10.35:3001/api/carga-trabajo/aplicar-sugerencias
```

EVIDENCIA THUNDER CLIENT



The screenshot displays the Thunder Client interface. The top bar shows the URL `192.168.10.35:3001/api/c...`. The main area is divided into two panels. The left panel, titled 'Query', shows the method 'POST' and the full URL `http://192.168.10.35:3001/api/carga-trabajo/aplicar-sugerencias`. Below this, there is a 'Query Parameters' section with a table for parameters. The right panel, titled 'Response', shows the status '200 OK', size '81 Bytes', and time '1.05 s'. Below this, the response body is displayed as a JSON object:

```
{  "ok": true,  "mensaje": "No hay operarios disponibles para reasignar",  "aplicados": 0}
```

parameter	value
-----------	-------

2.6 PATCH /api/carga-trabajo/umbrales

PATCH**/api/carga-trabajo/umbrales***Cambia los límites de sobrecarga y disponibilidad en caliente sin reiniciar el servidor.*

REQUEST

```
PATCH /api/carga-trabajo/umbrales HTTP/1.1
Host: 192.168.10.35:3001
x-api-key: texticode-2026
Content-Type: application/json

{
  "limite_sobrecarga": 3,
  "limite_disponible": 1
}
```

// Ambos campos son opcionales – se puede enviar solo uno.
// limite_sobrecarga: entero >= 1
// limite_disponible: entero >= 0

URL completa

```
http://192.168.10.35:3001/api/carga-trabajo/umbrales
```

EVIDENCIA THUNDER CLIENT

The screenshot shows the Thunder Client interface. The top bar indicates a PATCH request to `192.168.10.35:3001/`. The main area shows the request details: Method `PATCH`, URL `http://192.168.10.35:3001/api/carga-trabajo/umbrales`, and a `Send` button. The response is displayed on the right, showing a status of `200 OK`, size of `102 Bytes`, and time of `3 ms`. The response body is a JSON object: `{ "ok": true, "mensaje": "Umbrales actualizados", "umbrales": { "limite_sobrecarga": 8, "limite_disponible": 2 } }`. The left panel shows the request body in JSON format: `{ "limite_sobrecarga": 8, "limite_disponible": 2 }`.

The screenshot shows a REST client interface with a PATCH request to `http://192.168.10.35:3001/api/carga-trabajo/umbrales`. The request body is a JSON object with `"limite_sobrecarga": 0` and `"limite_disponible": 2`. The response is a 400 Bad Request with a message: `"limite_sobrecarga debe ser un entero positivo"`.

PATCH 192.168.10.35:3001/ X

PATCH `http://192.168.10.35:3001/api/carga-trabajo/umbrales` Send

Query Headers³ Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "limite_sobrecarga": 0,
3   "limite_disponible": 2
4 }
```

Status: 400 Bad Request Size: 70 Bytes Time: 3 ms

Response Headers⁷ Cookies Results Docs

```
1 {
2   "ok": false,
3   "mensaje": "limite_sobrecarga debe ser un entero positivo"
4 }
```

2.7 PATCH /api/carga-trabajo/fases/id/operario

PATCH**/api/carga-trabajo/fases/id/operario***Cambia el operario asignado a una orden específica.*

REQUEST

```
PATCH /api/carga-trabajo/ordenes/:id/operario HTTP/1.1
Host: 192.168.10.35:3001
x-api-key: texticode-2026
Content-Type: application/json

{
  "Id_Operario_Destino": 30
}

// :id -> Id_Orden (entero positivo)
// Id_Operario_Destino: operario activo existente
```

URL completa

```
http://192.168.10.35:3001/api/carga-trabajo/fases/id/operario
```

EVIDENCIA THUNDER CLIENT

The screenshot displays the Thunder Client interface. The top bar shows the URL `http://192.168.10.35:3001/api/carga-trabajo/fases/42/operario` and the method `PATCH`. The `Body` tab is selected, showing the JSON request: `{ "Id_Operario_Destino": 52 }`. The `Response` tab is also selected, showing the JSON response: `{ "ok": true, "mensaje": "Fase 4 de la orden #179 reasignada a Stevan Wagner", "data": { "tipo": "fase", "Id_Orden_Operario": 42, "Id_Orden": 179, "Numero_Fase": 4, "Producto": "Camisetas oversize ", "operario_anterior": 43, "operario_nuevo": { "id": 52, "nombre": "Stevan Wagner" } } }`. The status bar indicates `Status: 200 OK`, `Size: 259 Bytes`, and `Time: 1.12 s`.

192.168.10.35:3001/api/ca. X

PATCH http://192.168.10.35:3001/api/carga-trabajo/fases/4211/operario Send

Query Headers 3 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "Id_Operario_Destino": 52
3 }
```

Status: 404 Not Found Size: 68 Bytes Time: 1.03 s

Response Headers 7 Cookies Results Docs

```
1 {
2   "ok": false,
3   "mensaje": "No se encontró una fase activa con id 4211"
4 }
```

3. Diagrama Entidad-Relación

El módulo de carga de trabajo opera sobre 3 tablas en Supabase PostgreSQL. La tabla central es orden_produccion, que vincula operarios con sus órdenes activas.

Tabla	Campos
usuario	Id_Usuario (PK) · Nombre_Completo · Nombre_Usuario · Correo (UNIQUE) · Telefono · Estado · Id_Rol (FK)
rol	Id_Rol (PK) · Nombre_Rol (admin operario cliente)
orden_produccion	Id_Orden (PK) · Id_Operario (FK) · Id_Cliente (FK) · Id_Material (FK) · Producto · Cantidad · Prioridad · Estado · Unidades · Unidades_Realizadas · Fecha_Limite · Fecha_Creacion

DIAGRAMA (pegar imagen del ERD aquí)

[ESPACIO PARA PANTALLAZO]
Diagrama Entidad-Relación completo — pegar captura de dbdiagram / Supabase

Relaciones clave

Origen	Relación	Destino	Descripción
usuario	1 → N	orden_produccion	Un operario tiene múltiples órdenes asignadas
rol	1 → N	usuario	Un rol agrupa múltiples usuarios
usuario (admin)	1 → N	orden_produccion	El admin gestiona y reasigna las órdenes

4. Validaciones y Manejo de Errores

Todos los endpoints comparten el middleware apiKey.js y siguen una política uniforme de respuesta de errores.

Escenario	Validación en código	HTTP
API Key ausente	<code>!req.headers['x-api-key']</code>	401 Unauthorized
API Key incorrecta	<code>key !== process.env.API_KEY_EFICIENCIA</code>	403 Forbidden
ID no numérico	<code>isNaN(id)</code>	400 Bad Request
Operario no encontrado	<code>rows.length === 0</code>	404 Not Found
Orden no activa	<code>Estado NOT IN ('En Proceso', 'Pausado')</code>	404 Not Found
Campos requeridos faltantes	<code>!Id_Orden !Id_Operario_Destino</code>	400 Bad Request
Estado inválido en PATCH	<code>!['activo', 'inactivo'].includes(Estado)</code>	400 Bad Request
Array vacío en multiple	<code>!Array.isArray(movimientos) length===0</code>	400 Bad Request
<code>umbral_sobrecarga < 1</code>	<code>!Number.isInteger(v) v < 1</code>	400 Bad Request
Error interno de BD	<code>catch(err) -> log interno</code>	500 Server Error

Middleware apiKey.js

```
const verificarApiKey = (req, res, next) => {
  const key = req.headers['x-api-key']
  if (!key)
    return res.status(401).json({ ok: false,
      mensaje: 'Acceso denegado. Se requiere API Key en el header x-api-key' })
  if (key !== process.env.API_KEY_EFICIENCIA)
    return res.status(403).json({ ok: false, mensaje: 'API Key invalida' })
  next()
}

export default verificarApiKey
```